To access an object farther down the tree, you can use another dot to *refine* the selection:

```
response.results['23443'].user
# "Guard"

response.results['23445'].text
# "We found them."

response.results['23446']
# {user: "Guard", text: "Found them? In Mercia?! The coconut\'s
tropical!"}
```

Wait a minute!

If you’re thinking that a chain of object calls is a train wreck, then you’re correct in the case of a method call returning another object that is then modified. In this case, though, the object is being used purely as data. All of the data is self-contained, and there’s no indirection occurring.

You’ve seen CoffeeScript objects used as data and transformed with comprehensions. You’ll need to do more with objects, though, and in CoffeeScript this means your objects will need to work with your functions. One piece of glue used with objects and functions is the binding of functions to objects.

4.5. Binding

When working with objects, you’ll need some way to refer to them. One important way this is achieved in CoffeeScript is with binding. Binding in CoffeeScript works the same as in JavaScript, with the important addition of new syntax that makes it easier to deal with.

If you’re coming from other programming languages, the way binding works in JavaScript and CoffeeScript can be counter to your expectations. For that reason, it’s important to start at the beginning and take it slowly.